

MyRA Personal AI Assistant - Master Interview Preparation Document

Generated on: 2026-07-08

Project root: /Users/cherry/projects/personal-ai-assistant

This document is based on the actual repository files inspected in this project. It focuses heavily on the backend, RAG, agents, APIs, architecture, database design, and interview explanations. The frontend is also covered so you can explain the complete product flow.

Table of Contents

- A. Project Overview
- B. Complete Project Architecture
- C. Backend Deep Explanation
- D. Database and Data Model
- E. RAG System Detailed Explanation
- F. Agentic AI Flow Detailed Explanation
- G. LangChain, LangGraph, and AI Libraries
- H. Frontend Explanation
- I. Complete Feature-by-Feature Flow
- J. Tech Stack and Tools
- K. Design Decisions
- L. Bugs, Gaps, and Production Improvements
- M. Interview Question Bank
- N. File-by-File Revision Map

A. Project Overview

What this project does

MyRA is a personal AI assistant that connects to a user's Google account, syncs personal Gmail and Google Calendar data, stores that data as searchable documents, creates embeddings, and answers questions using RAG. It also has agentic workflows for creating calendar events and safely drafting/sending emails.

The project has two main applications:

- Backend: backend, an Express 5 Node.js service with PostgreSQL, pgvector-style vector search, Google OAuth, Gmail/Calendar integrations, RAG services, LangChain model calls, and LangGraph agents.
- Frontend: frontend, a Vite React app with Zustand stores, a custom SPA router in App.jsx, chat UI, auth callback handling, profile/data sync UI, stats dashboard, and settings/profile screens.

Main problem it solves

The problem is that personal information is spread across inboxes and calendars. A user may know that something exists in their email or calendar but not remember where. MyRA lets the user ask natural-language questions like: What did Rahul send me last week? What meetings do I have today? Draft an email to Priya about the review. Schedule a meeting tomorrow at 3 PM.

Users

The expected user is a person who wants a private assistant over personal productivity data. In interview terms, the user is a knowledge worker who spends time in Gmail and Calendar and wants retrieval, summarization, drafting, and scheduling from one chat interface.

Major features

- Google OAuth login and account connection.
- Encrypted storage of Gmail and Google Calendar access/refresh tokens.
- Gmail sync into a unified documents table.
- Google Calendar sync into the same unified documents table.
- RAG over synced personal data using chunking, embeddings, vector search, context building, prompt construction, and LLM response generation.
- Conversation memory stored in PostgreSQL.
- Intent routing between general chat, RAG, calendar agent, and email agent.
- Calendar creation agent with missing-field collection, conflict checking, alternative-slot suggestion, and confirmation before event creation.
- Email drafting agent with recipient resolution, draft approval/edit loop, and a revoke window before sending.
- Stats dashboard for emails, tokens, cost, chat sessions, and calendar events.
- Socket.IO support for sync progress updates.
- Profile page for display name, Gmail/Calendar sync, and sync history.

How the parts work together

The frontend calls REST APIs in the backend. The backend authenticates the user through Google OAuth and stores Google tokens in `api_credentials`. When the user starts a sync, the backend fetches Gmail or Calendar items through Google APIs, normalizes them, stores them in documents, then creates chunks and embeddings in `document_chunks`. When the user asks a question, the backend classifies intent. For personal-data questions, it embeds the query, retrieves relevant chunks, builds context, sends that context to an LLM, saves the conversation, and returns an answer with sources. For action requests, the backend runs a LangGraph agent that asks for confirmation before changing external state.

One-minute interview answer

My project is MyRA, a personal AI assistant for Gmail and Google Calendar. The backend is an Express and PostgreSQL system. Users sign in with Google OAuth, and I store encrypted OAuth tokens so the backend can sync Gmail and Calendar data. I normalize emails and events into a common documents table, split them into chunks, generate embeddings, and store chunk vectors for semantic retrieval. When the user asks a question, the chat endpoint routes the message using an intent classifier. Personal-data questions go through my RAG pipeline: embed query, vector search, build context, prompt the LLM, save the conversation, and return sources. I also built agentic flows with LangGraph: one calendar agent that collects event details, checks conflicts, asks for confirmation, and creates events, and one email agent that drafts an email, lets the user review or edit it, and gives a short revoke window before sending through Gmail.

Three-minute detailed interview answer

MyRA is a full-stack personal AI assistant. The frontend is a Vite React app with Zustand state stores for auth, chat, and sync. The backend is more important: it is an Express 5 app with route/controller/service/repository layers. Google OAuth is used for authentication and authorization. After OAuth callback, the backend creates or updates the user row, encrypts Google tokens using AES-256-CBC in `GoogleAuthService`, stores credentials for both Gmail and Calendar, then issues a JWT for the frontend.

For RAG, I built an ingestion pipeline and a query pipeline. Ingestion pulls Gmail messages and Calendar events using Google APIs. Normalizers convert source-specific API data into a unified document shape: source, type, content, title, timestamp, author, and metadata. Documents are stored in PostgreSQL. Pending documents are split using LangChain's `RecursiveCharacterTextSplitter` with chunk size 2700 and overlap 400. The chunks are embedded using OpenAI embeddings and inserted into `document_chunks` with pgvector-compatible vectors.

For querying, the chat API first classifies intent. If it is a personal-data query, the RAG pipeline embeds the query, searches `document_chunks` scoped by user id, filters results by vector distance, builds a source-numbered context block, adds conversation history, calls the selected chat model, saves the conversation, and returns both answer and source documents. For general questions it skips RAG and calls the LLM directly.

I also implemented agentic flows using LangGraph. The calendar agent extracts event details, asks for missing fields, checks Google Calendar conflicts, suggests free slots, asks for confirmation, then creates the event. The email agent uses LangGraph interrupts to pause for human choices: selecting a recipient, approving or editing a draft, and revoking a pending send. This is important because actions like sending email and creating events need human confirmation and safety checks.

B. Complete Project Architecture

High-level architecture

React/Vite frontend
-> REST fetch calls to Express backend
-> Socket.IO sync progress listeners on ProfilePage

Express backend
-> Auth routes and Google OAuth
-> Chat routes
 -> intent router
 -> general LLM
 -> RAG pipeline
 -> calendar LangGraph agent
 -> email LangGraph agent
-> Sync routes
 -> Gmail/Calendar data sources
 -> normalizers
 -> document repository
 -> embedding pipeline
-> Stats routes
 -> aggregation queries

PostgreSQL
-> users
-> api_credentials
-> documents
-> document_chunks
-> conversations
-> sync_logs
-> llm_usage_logs
-> embedding_costs
-> recipients

Folder structure explanation

Important backend folders:

- backend/index.js: process entrypoint. Loads env, connects to DB, starts Express, attaches Socket.IO, and handles SIGTERM/SIGINT shutdown.
- backend/src/app.js: Express app setup, CORS, body parsing, static files, route mounting.
- backend/src/api/routes: route definitions for auth, chat, sync, stats, and an unmounted calendar-agent route.
- backend/src/api/controllers: controller classes/functions that validate requests, call services, and shape responses.
- backend/src/database: repository classes for PostgreSQL access.
- backend/src/RAG: RAG ingestion, retrieval, context, prompt, memory, LLM, and top-level RAG service.
- backend/src/agent: intent router and LangGraph agent implementations.
- backend/src/service: Google data sources, normalizers, OAuth, email sending, WebSocket, cron, and alert services.
- backend/src/utills: logger, validation, token utilities, mailer, constants, exchange-rate helper, and email templates.
- backend/test: ad-hoc smoke tests, not a full automated test suite.

Important frontend folders:

- frontend/src/App.jsx: custom SPA routing, theme handling, auth check, layout selection.
- frontend/src/api: REST clients for auth, chat, sync, stats, home, and user profile updates.
- frontend/src/store: Zustand stores for auth, chat, and sync.
- frontend/src/pages: login, auth callback, home, chat, profile, stats, settings.

- frontend/src/components/chat: chat window, assistant/user messages, typing indicator, placeholder ChatInput.
- frontend/src/components/layout/Sidebar.jsx: chat history, navigation, profile menu, and calendar sync button.
- frontend/src/service/socketService.js: Socket.IO client wrapper used by ProfilePage.

Backend architecture

The backend follows a practical layered architecture:

- Routes define HTTP paths and attach middleware.
- Controllers own request/response behavior.
- Services own external integrations and business pipelines.
- Repositories own SQL queries.
- RAG and agents are separated into their own modules because they are complex workflows.

This structure is acceptable for a junior-to-mid project because it avoids putting all logic inside route handlers. It also keeps the interview story clean: routes -> controllers -> services -> repositories.

Frontend architecture

The frontend is a single-page React app but does not use React Router. Instead, App.jsx maps URL paths to page ids using PATH_TO_PAGE and PAGE_TO_PATH, listens to popstate, and updates browser history manually. State is kept in Zustand stores. API clients are plain fetch wrappers. The UI is custom CSS with some Tailwind installed but not used as the main styling system.

Database architecture

The code expects PostgreSQL. Vector operations use the pgvector <=> distance operator. There are no migrations in the repo, so table structure must be inferred from SQL. Main tables are users, api_credentials, documents, document_chunks, conversations, sync_logs, llm_usage_logs, embedding_costs, recipients, and optionally agent_checkpoints.

AI/RAG architecture

RAG has two halves:

- Ingestion: Google API raw items -> normalizer -> documents -> chunker -> embeddings -> document_chunks.
- Query: user query -> intent classifier -> query embedding -> vector search -> context builder -> prompt builder -> LLM -> saved conversation -> frontend response.

Agent architecture

There are two LangGraph agents:

- Calendar agent in backend/src/agent/calenderAgent: state graph with nodes for parsing, asking missing info, checking conflicts, suggesting slots, awaiting confirmation, and creating events.
- Email agent in backend/src/agent/emailAgent: state graph with interrupts for recipient selection, draft approval/editing, and pending-send revoke.

The calendar folder is spelled calenderAgent in the repo. The standalone route path is also spelled /agent/calender.

External integrations

- Google OAuth: login and offline access.
- Gmail API: reading synced messages and sending emails.
- Google Calendar API: reading events, checking free/busy, creating events.
- OpenAI: embeddings, chat, intent routing, and email draft structured output depending on env config.
- Anthropic: calendar-agent extraction and optional chat provider.
- Frankfurter exchange-rate API: converts USD model pricing to INR in usdToInr.
- SMTP/Nodemailer: sends cost-alert emails.
- Socket.IO: emits sync progress to the browser.

Request lifecycle

A typical RAG request lifecycle:

1. User sends a message from ChatWindow.jsx.
2. useChatStore.sendMessage appends the user message and calls chatApi.sendMessage.
3. POST /chat/message runs optional JWT auth.
4. ChatController.sendMessage validates the message and resolves the handler.
5. routeIntent calls LLMService.generateResponse to classify the message.
6. For rag, RagChain.chat calls QueryPipeline.run.
7. Retriever embeds the query and searches document_chunks by vector distance.
8. buildContext creates source-numbered context.
9. MemoryService loads conversation history.
10. buildPrompt creates model messages.
11. LLMService calls OpenAI or Anthropic and logs token/cost usage.
12. MemoryService.saveConversation inserts into conversations.
13. Controller returns answer and sources.
14. Zustand store appends the assistant message.

Why this architecture is used

This architecture is good for this project because it separates source ingestion from query-time RAG, separates RAG from action-taking agents, and keeps external API logic outside controllers. It is also easy to explain: sync builds the knowledge base; chat queries the knowledge base; agents perform actions with confirmation.

Alternative architectures

- Use Next.js full-stack instead of separate frontend/backend. Simpler deployment, but less explicit backend separation.

- Use a managed vector database such as Pinecone, Weaviate, or Qdrant. Easier vector operations at scale, but more infrastructure and cost.
- Use Redis queues or BullMQ for background sync. More reliable than fire-and-forget promises, but more setup.
- Use Prisma migrations and ORM. More maintainable schema evolution, but raw SQL currently gives direct control.
- Use server-side sessions instead of JWT in localStorage. More secure for browser apps, but requires session store and cookie strategy.
- Use a single agent for all tasks. More flexible, but harder to control and harder to prove safe for email/calendar side effects.

Why the current approach is acceptable

For an interview project, the current approach is acceptable because it demonstrates backend fundamentals, OAuth, database repositories, vector retrieval, LLM orchestration, and agent safety flows. The production gaps are mostly around auth hardening, migrations, background job durability, persistent LangGraph checkpoints, and route cleanup.

C. Backend Deep Explanation

Backend framework used

The backend uses Express 5.2.1 with ES modules. The package is myra-server in backend/package.json. Development uses tsx watch index.js, and TypeScript compilation is configured with allowJs: true and checkJs: false in backend/tsconfig.json.

Why Express was chosen

Express is lightweight and flexible. It is a good fit because this project needs custom workflows rather than a rigid MVC framework. Express lets the code mount route modules for auth, sync, chat, and stats while keeping complex logic in separate services.

Server startup

backend/index.js does the following:

- Imports ./src/config/env.js, which currently only imports dotenv/config.
- Imports logger, socketServer, app, and connectToDB.
- Resolves PORT from process.env.PORT || 2020.
- Calls connectToDB() before listening.
- Starts Express and attaches Socket.IO with socketServer.initialize(server).
- Handles SIGTERM and SIGINT by closing the HTTP server.

Express app setup

backend/src/app.js configures:

- JSON body parsing with a small 16kb limit.
- URL encoded body parsing.
- Static file serving from public.
- CORS origins from CORS_ORIGIN, FRONTEND_URL, or default http://localhost:5173.
- Credentials allowed.
- Methods GET, POST, PUT, DELETE, PATCH.
- Routes mounted at /auth, /sync, /chat, and /stats.

Important: backend/src/api/routes/agent.js exists but is not mounted in app.js.

Configuration and environment variables

Main environment variables used by code:

- PORT: backend port, default 2020 in index.js.
- FRONTEND_URL or CORS_ORIGIN: allowed browser origin and OAuth redirect target.

- DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD: required by environment.js and dbConfig.ts.
- DB_SSL, DB_MAX_CONNECTIONS: included in config object but dbConfig.ts does not use DB_SSL.
- JWT_SECRET: signs and verifies app JWTs.
- GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI: Google OAuth.
- TOKEN_ENCRYPTION_KEY: hex key used for AES-256-CBC token encryption. The first 64 hex chars are used.
- OPENAI_API_KEY, OPENAI_CHAT_MODEL, OPENAI_MODEL_TEMP, OPENAI_MAX_TOKENS, OPENAI_EMBEDDING_MODEL, EMBEDDING_DIMENSIONS, OPENAI_LIGHT_MODEL.
- ANTHROPIC_API_KEY, ANTHROPIC_CHAT_MODEL, ANTHROPIC_MODEL_TEMP, ANTHROPIC_MAX_TOKENS.
- SYNC_USER_ID: fallback user id when auth is optional or absent.
- MAIL_USER, MAIL_APP_PASSWORD, MAIL_SMTP_HOST, MAIL_SMTP_PORT, MAIL_FROM_NAME, MAIL_FROM_ADDRESS, MAIL_ALERT_RECIPIENT.
- ANTHROPIC_MONTHLY_BUDGET, GOOGLE_MONTHLY_BUDGET, CREDITS_ALERT_CRON_SCHEDULE, ENABLE_CREDITS_ALERT_CRON.

backend/src/config/environment.js validates DB env vars at module load and exits the process if they are missing.
 backend/src/config/dbConfig.ts lazily creates the pg Pool in getPool() so dotenv has time to load before the pool is created.

Database connection

connectToDB calls getPool().connect(), logs success, and releases the client. The pool has connectionTimeoutMillis: 5000 and max: 10. The repository files often import pool directly. Because ES module imports are live bindings, after getPool() assigns the pool the repositories can use it, but this pattern is fragile if code tries to query before DB connection.

Routes and controllers

The route modules are:

- authRoutes.js: Google login, callback, current user, update display name, logout.
- chat.js: chat message, planned stream message, conversation id, conversation list, history, email status.
- syncRoutes.js: Gmail sync, Calendar sync, sync status, sync history.
- stats.js: all stats.
- agent.js: standalone calendar-agent route, currently not mounted.

Authentication and authorization

Authentication is inconsistent:

- Auth-specific routes manually validate JWTs in AuthController.getCurrentUser and updateUserName.
- Chat and stats use local optionalAuth middleware. If a valid token exists, req.user is set. If missing, the code falls back to SYNC_USER_ID. In chat, an invalid token returns 401. In stats, an invalid token is ignored.
- Sync routes have no auth middleware, even though syncing reads personal data and uses userId from the request body.

Interview answer: This is okay for a prototype, but production should require authentication on all personal-data routes, derive `userId` from the JWT instead of the body, and remove `SYNC_USER_ID` fallback outside local development.

Error handling

Error handling is local to controllers and services. There is no global Express error middleware in `app.js`. Controllers generally return JSON 400/401/404/500. Some auth errors redirect to frontend login. Background sync errors are logged, written to `sync_logs`, and emitted by `WebSocket`.

Validation

Validation exists but is manual:

- Chat validates that message is a non-empty string.
- Auth update validates non-empty `userName`.
- Sync validates `userId` presence.
- Repository helpers validate some numeric inputs.
- `utils/validation.js` validates unified documents and source metadata, but current ingestion normalizers mostly bypass `schemas/index.js` and call `DocumentRepository.create` directly.
- Email agent uses Zod structured output schemas for parsing requests and drafts.

There are no centralized DTO schemas for API request bodies. Adding Zod route schemas would make the backend safer.

Logging

`backend/src/utils/logger.ts` defines a simple logger with `ERROR`, `WARN`, `INFO`, and `DEBUG` levels. It writes colored logs to console. It also has sync-specific helper methods. Several files import `logger.js` even though the source file currently exists as `logger.ts` in the dirty worktree. TypeScript/tsx may resolve this during development, but plain Node over source JS would not.

WebSocket service

`backend/src/service/websocket/socketService.js` creates a `Socket.IO` server. It stores connected clients in memory, accepts an `identify` event, and emits events like `sync:gmail:progress`, `sync:gmail:complete`, `sync:google_calendar:error`, `rag:progress`, and system health events. In current active code, sync progress is the main `WebSocket` usage. The RAG progress emitters exist but `QueryPipeline` does not call them.

Background jobs and cron

`CredsAlertCronJob` checks monthly LLM usage and sends alert emails when usage crosses 50, 75, or 90 percent of configured budgets. It uses `StatsRepository.getLLMCredsUsage` and `CredsAlertService.checkAndAlert`. Alerts are deduplicated in an in-memory `Map`.

`CronManager` is incomplete: it only creates `credsAlert`, but `startAll`, `stopAll`, and `getAllStatus` also reference `embedding`, `gmailSync`, and `calendarSync`, which are not defined. Also, `index.js` does not start `CronManager`.

File upload handling

There is no active file upload handling. The chat composer shows an attach icon, but no upload API exists.

Complete backend API documentation

GET / - Health check

- Purpose: Returns a simple string to prove the Express server is running.
- Request params/body: No body and no params.
- Response body: Plain text: API is running...
- Error cases: Only framework/server errors.
- Authentication: No authentication.
- Controller/function: Inline route in backend/src/app.js.
- Services/business logic: None.
- Database tables/collections: None.
- Example request: curl http://localhost:2020/
- Example response: API is running...
- Interview explanation: Explain it as a basic operational endpoint used to quickly verify that the backend process started and is reachable.

GET /auth/google/login - Start Google OAuth login

- Purpose: Builds a Google OAuth consent URL for Gmail, Gmail send/compose, Google Calendar events, profile, and email scopes.
- Request params/body: No body. The backend creates a random state string but currently does not persist or validate it later.
- Response body: JSON with success true and data.authUrl.
- Error cases: Returns through Express next(error) if URL generation fails.
- Authentication: No existing app auth required.
- Controller/function: AuthController.initiateGoogleLogin in backend/src/api/controllers/authController.js.
- Services/business logic: Uses google.auth.OAuth2 from googleapis.
- Database tables/collections: None at this step.
- Example request: GET /auth/google/login
- Example response: {"success":true,"data":{"authUrl":"https://accounts.google.com/o/oauth2/v2/auth?..."}}
- Interview explanation: Say this endpoint starts the account-connection flow. It requests offline access so the app can refresh tokens and later read Gmail, send Gmail, and read/write Calendar for the user.

GET /auth/google/callback - Google OAuth callback

- Purpose: Handles Google's callback, exchanges the authorization code for tokens, fetches Google profile data, creates or updates the user, stores encrypted credentials, signs a JWT, and redirects to the frontend callback page.
- Request params/body: Query params: code from Google, or error if the user denied consent.

- Response body: Redirects to FRONTEND_URL/auth/callback?token=<jwt> on success. Redirects to /login with error codes on failure.
- Error cases: access_denied, missing_code, auth_failed redirect paths. Internal errors are logged.
- Authentication: Google authorization code is required, but no existing app JWT is required.
- Controller/function: AuthController.handleGoogleCallback and AuthController.createOrUpdateUser.
- Services/business logic: googleapis OAuth2 and OAuth userinfo. GoogleAuthService.encrypt for token encryption.
- Database tables/collections: users and api_credentials.
- Example request: GET /auth/google/callback?code=4/0Ab...
- Example response: HTTP 302 redirect to http://frontend/auth/callback?token=<jwt>
- Interview explanation: Explain this as the bridge between Google identity and your own app identity. Google returns the code, the backend stores protected tokens, then the frontend receives only the app JWT.

GET /auth/me - Get current user

- Purpose: Validates the JWT, loads the user row, and adds connected source information from stored credentials.
- Request params/body: Authorization: Bearer <jwt>.
- Response body: JSON with data.user including user fields and connectedSources.
- Error cases: 401 for missing token, 401 for invalid token, 404 if the user id from the token is not found.
- Authentication: Required.
- Controller/function: AuthController.getCurrentUser.
- Services/business logic: jsonwebtoken, UserRepository, CredentialRepository.
- Database tables/collections: users and api_credentials.
- Example request: GET /auth/me with Authorization header.
- Example response:
{"success":true,"data":{"user":{"id":3,"email":"user@example.com"},"connectedSources":[{"source":"gmail"}]}}
- Interview explanation: Say this endpoint is how the frontend restores a login session from localStorage. It also tells the UI which integrations are connected.

PATCH /auth/user/name - Update display name

- Purpose: Updates user_name, which is the app-level display name separate from the Google account name.
- Request params/body: Authorization header plus JSON body { userName: string }.
- Response body: Updated user object in data.user.
- Error cases: 401 for missing/invalid token, 400 when userName is missing or blank.
- Authentication: Required.
- Controller/function: AuthController.updateUserName.
- Services/business logic: UserRepository.updateUserName.
- Database tables/collections: users.
- Example request: PATCH /auth/user/name {"userName":"Cherry"}
- Example response: {"success":true,"data":{"user":{"id":3,"user_name":"Cherry"}}}
- Interview explanation: Explain that it prevents overwriting the Google profile name and keeps a user-controlled app display name.

POST /auth/logout - Logout

- Purpose: Returns a logout success message. The frontend removes the JWT from localStorage.
- Request params/body: Optional Authorization header. No body required.
- Response body: {"success":true,"message":"Logged out successfully"}
- Error cases: Only unexpected server errors.
- Authentication: Not strictly enforced by the controller.
- Controller/function: AuthController.logout.
- Services/business logic: None.
- Database tables/collections: None.
- Example request: POST /auth/logout
- Example response: {"success":true,"message":"Logged out successfully"}
- Interview explanation: Say logout is currently client-side token removal plus a backend acknowledgement. There is no server-side JWT blacklist yet.

POST /chat/message - Send chat message

- Purpose: Main assistant endpoint. It routes the message to email agent, calendar agent, RAG, or general LLM chat.
- Request params/body: JSON body: message, conversationId, llmProvider, confirmationStatus, agentActive, activeAgentMode. JWT is optional; fallback user id comes from SYNC_USER_ID.
- Response body: JSON with queryId, conversationId, query, response, mode, agentActive flags, context and metadata. Email mode may return structured emailResponse.
- Error cases: 400 if message is missing. 401 if a supplied Bearer token is expired/invalid. 500 if routing, agent, RAG, or LLM flow fails.
- Authentication: Optional authentication middleware. For production this should be required because personal data is involved.
- Controller/function: chatController.sendMessage in backend/src/api/controllers/chatController.js.
- Services/business logic: routeIntent, calendarAgentGraph, invokeEmailAgent, RagChain, LLMService, MemoryService.
- Database tables/collections: conversations, documents, document_chunks, llm_usage_logs, recipients, api_credentials, Google Calendar/Gmail API when agents run.
- Example request: {"message":"What emails did I get from Rahul?","conversationId":null,"llmProvider":"OpenAI"}
- Example response: {"success":true,"conversationId":"uuid","response":"According to Source 1...","mode":"rag","context":{"selectedDocuments":2}}
- Interview explanation: Explain this as the orchestration endpoint. The backend first classifies intent, then chooses the correct path: RAG for personal-data Q&A, calendar graph for event creation, email graph for safe email drafting/sending, or a normal model call for general chat.

POST /chat/message/stream - Stream chat message

- Purpose: Intended streaming version of chat.
- Request params/body: Same concept as /chat/message.
- Response body: Would be SSE events if implemented.
- Error cases: Current route calls chatController.sendMessageStream, but that method is commented out in the controller. Calling this endpoint will fail at runtime.

- Authentication: Optional auth middleware is present in route.
- Controller/function: Route in backend/src/api/routes/chat.js points to a missing method.
- Services/business logic: Not active.
- Database tables/collections: Not active.
- Example request: POST /chat/message/stream
- Example response: Runtime failure because sendMessageStream is not available.
- Interview explanation: Be honest: streaming was planned and partially sketched, but the current RAG service is non-streaming and the active route should be removed or implemented.

POST /chat/conversation - Create conversation id

- Purpose: Returns a new UUID so the frontend can start a conversation before the first message is saved.
- Request params/body: No body.
- Response body: data.conversationId and createdAt.
- Error cases: 500 if UUID generation or response fails, which is unlikely.
- Authentication: No authentication on this route.
- Controller/function: chatController.createConversation.
- Services/business logic: uuid.
- Database tables/collections: None.
- Example request: POST /chat/conversation
- Example response: {"success":true,"data":{"conversationId":"uuid","createdAt":"2026-07-08T..."}}
- Interview explanation: Say the real conversation record is created when a message is saved; this only reserves an id.

GET /chat/conversations - List conversations

- Purpose: Returns recent distinct conversation threads for the sidebar.
- Request params/body: Optional query limit. Optional Bearer token; otherwise falls back to SYNC_USER_ID.
- Response body: data.conversations with conversationId, title, startedAt, lastMessageAt.
- Error cases: 500 on DB failure. 401 only if an invalid Bearer token is supplied.
- Authentication: Optional in code; should be required in production.
- Controller/function: chatController.getConversations.
- Services/business logic: ConversationRepository.getConversations.
- Database tables/collections: conversations.
- Example request: GET /chat/conversations?limit=50
- Example response: {"success":true,"data":{"conversations":[{"conversationId":"uuid","title":"What emails..."}]}}
- Interview explanation: Explain that the title is derived from the first user message and ordering uses the most recent message timestamp.

GET /chat/history/:conversationId - Get conversation history

- Purpose: Loads previous turns for a conversation and formats them for the UI.
- Request params/body: Path param conversationId. Optional query limit. Optional auth.

- Response body: data.history as pairs of user_message and assistant_message.
- Error cases: 400 if conversationId is missing, 500 on DB failure.
- Authentication: Optional in code; user id is still scoped from token or SYNC_USER_ID.
- Controller/function: chatController.getHistory.
- Services/business logic: MemoryService.loadHistory.
- Database tables/collections: conversations.
- Example request: GET /chat/history/abc?limit=50
- Example response:


```
{"success":true,"data":{"conversationId":"abc","history":[{"user_message":"Hi","assistant_message":"Hello"}]}}
```
- Interview explanation: Say this converts the DB memory format back into UI message pairs. It currently assumes user and assistant messages alternate.

GET /chat/email-status/:conversationId - Get email workflow status

- Purpose: Reads the LangGraph email-agent checkpoint state for the conversation.
- Request params/body: Path param conversationId. Optional auth.
- Response body: exists, active, emailStatus, response, interrupt, revokeDeadline.
- Error cases: 400 if conversationId is missing, 500 on state read failure.
- Authentication: Optional in code.
- Controller/function: chatController.getEmailStatus.
- Services/business logic: getEmailSessionStatus in backend/src/agent/emailAgent/index.js.
- Database tables/collections: No DB table because the current email graph uses MemorySaver in process memory.
- Example request: GET /chat/email-status/abc
- Example response: {"success":true,"conversationId":"abc","exists":true,"active":true,"emailStatus":"pending_revoke"}
- Interview explanation: Explain it lets the UI refresh pending email send state, especially during the short revoke window.

POST /sync/gmail - Start Gmail sync

- Purpose: Starts background ingestion from Gmail and then embedding generation for that user's pending documents.
- Request params/body: JSON body: userId required, syncType optional full or incremental. sinceDate and untilDate are accepted but not used.
- Response body: Immediate response with syncId and status running. The actual work continues after the response.
- Error cases: 400 if userId is missing. 500 if sync log creation or initial setup fails. Later failures are written to sync_logs and emitted by WebSocket.
- Authentication: No auth middleware on the route even though frontend sends JWT. This is a production gap.
- Controller/function: SyncController.syncGmail and performDocumentsSync.
- Services/business logic: IngestionPipeline, GmailDataSource, GmailNormalizer, EmbeddingPipeline, socketServer.
- Database tables/collections: sync_logs, documents, document_chunks, api_credentials.
- Example request: {"userId":3,"syncType":"incremental"}
- Example response: {"success":true,"data":{"syncId":22,"status":"running","message":"Gmail sync started"}}

- Interview explanation: Explain the async design: the API returns fast, while the backend emits progress and continues ingestion plus embedding in the background.

POST /sync/calendar - Start Calendar sync

- Purpose: Starts background ingestion from Google Calendar and embedding generation.
- Request params/body: JSON body: userId required, syncType optional full or incremental.
- Response body: Immediate syncId and running status.
- Error cases: 400 if userId missing, 500 for initial failures, WebSocket and sync_logs for background errors.
- Authentication: No auth middleware on the route.
- Controller/function: SyncController.syncCalendar and performDocumentsSync.
- Services/business logic: IngestionPipeline, GoogleCalendarDataSource, GoogleCalendarNormalizer, EmbeddingPipeline, socketServer.
- Database tables/collections: sync_logs, documents, document_chunks, api_credentials.
- Example request: {"userId":3,"syncType":"full"}
- Example response: {"success":true,"data":{"syncId":23,"status":"running","message":"Calendar sync started"}}
- Interview explanation: Explain it mirrors Gmail sync but uses Google Calendar events and event-specific normalization.

GET /sync/status/:syncId - Get sync status

- Purpose: Reads one sync_logs row.
- Request params/body: Path param syncId.
- Response body: data is the sync log row.
- Error cases: 404 when the sync id is not found, 500 on DB failure.
- Authentication: No auth middleware.
- Controller/function: SyncController.getSyncStatus.
- Services/business logic: SyncLogRepository.findById.
- Database tables/collections: sync_logs.
- Example request: GET /sync/status/22
- Example response: {"success":true,"data":{"id":22,"status":"success"}}
- Interview explanation: Say it is the polling fallback to WebSocket progress.

GET /sync/history - Get sync history

- Purpose: Returns recent sync logs for a user and source.
- Request params/body: Query params: userId required, source defaults to gmail, limit defaults to 10.
- Response body: data.history array.
- Error cases: 400 if userId missing, 500 on DB failure.
- Authentication: No auth middleware.
- Controller/function: SyncController.getSyncHistory.
- Services/business logic: SyncLogRepository.findBySource.
- Database tables/collections: sync_logs.

- Example request: GET /sync/history?userId=3&source=gmail&limit=10
- Example response: {"success":true,"data":{"history":[{"source":"gmail","status":"success"}]}}
- Interview explanation: Explain that the profile page uses it to show sync history.

GET /stats/all - Stats dashboard

- Purpose: Aggregates data for the frontend stats page: emails, tokens, sessions, calendar events, and model cost.
- Request params/body: Query range is one of 7d, 14d, 30d, 90d. Optional Bearer token; invalid token is ignored and fallback user id is used.
- Response body: data.emails, data.tokens, data.reminders, data.cost, data.sessions, data.calEvents.
- Error cases: 500 if aggregation queries fail.
- Authentication: Optional in code.
- Controller/function: StatsController.getAllStats.
- Services/business logic: StatsRepository.
- Database tables/collections: llm_usage_logs, embedding_costs, conversations, documents.
- Example request: GET /stats/all?range=14d
- Example response: {"success":true,"data":{"emails":[2,3],"tokens":[{"name":"model","value":1000}], "cost":[{"provider":"OpenAI","spend":0.4}]}}
- Interview explanation: Say this endpoint denormalizes several backend metrics into the exact shape needed by the dashboard.

POST /agent/calender - Calendar agent route

- Purpose: Standalone calendar agent route that directly invokes calendarAgentGraph.
- Request params/body: Body: message, threadId, confirmationStatus.
- Response body: response, status, eventDetails, pendingConfirmation.
- Error cases: 500 on agent failure.
- Authentication: No auth shown in route.
- Controller/function: backend/src/api/routes/agent.js.
- Services/business logic: calendarAgentGraph.
- Database tables/collections: Google Calendar API through graph nodes.
- Example request: {"message":"Schedule a meeting tomorrow at 3","threadId":"t1"}
- Example response: {"response":"What should the event be called?","pendingConfirmation":false}
- Interview explanation: Mention this route currently is not mounted in app.js, so the active calendar-agent path is through /chat/message, not this standalone route.

D. Database and Data Model

Database used

The backend uses PostgreSQL through the pg package. Vector search uses the pgvector operator `<=>`, so the database needs the pgvector extension and vector columns.

Why PostgreSQL is a good choice here

PostgreSQL is acceptable because the project needs relational data such as users, credentials, documents, conversations, sync logs, and usage logs. With pgvector, the same database can store vector embeddings and run similarity search. For a personal assistant prototype, that keeps the architecture simpler than combining Postgres plus a separate vector database.

Missing migrations

There are no migration files in this repo. This is a major setup gap. The table design below is inferred from repository SQL and service code. In production, you should add migrations using Prisma, Drizzle, Knex, node-pg-migrate, or raw SQL migration files.

Inferred tables

users

Used by UserRepository. Important fields:

- id: internal user id.
- google_id: Google profile id.
- email: Google account email.
- name: Google profile name.
- user_name: app-level display name that can be edited independently.
- picture: Google profile picture URL.
- email_verified, locale, preferences, status.
- last_login_at, login_count, created_at, updated_at.

Relationships: users.id is referenced by documents.user_id, conversations.user_id, sync_logs.user_id, api_credentials.user_id, llm_usage_logs.user_id, and recipients.user_id.

api_credentials

Used by CredentialRepository and GoogleAuthService. Important fields:

- id: credential row id.
- user_id: owner.
- source: gmail or google_calendar.
- access_token: encrypted token.

- refresh_token: encrypted refresh token.
- token_expires_at: expiry timestamp.
- scope or scopes: scopes from Google.
- created_at, updated_at.

There is inconsistency in the repository: older methods use scope and conflict on (source), while OAuth storage uses scopes and conflict on (user_id, source). The current OAuth path uses storeOAuthTokens, so the schema should support scopes and a unique (user_id, source) index.

documents

Used by DocumentRepository, EmbeddingRepository, stats queries, and ingestion. Important fields:

- id: internal DB id.
- user_id: owner.
- document_id: external unique id such as gmail_<messageId> or calendar_<eventId>.
- source: gmail or calendar.
- type: email or event.
- content: normalized full text.
- title: subject or event summary.
- timestamp: source occurrence time.
- author: email sender or calendar organizer.
- metadata: JSONB source metadata.
- needs_embedding: boolean used by EmbeddingPipeline.
- embedding, embedding_generated_at, embedding_tokens, embedding_model: older document-level embedding support still present in EmbeddingRepository.
- created_at, updated_at.

Current active RAG stores chunk embeddings in document_chunks, but some repository code still supports document-level embeddings. This is a design transition to mention honestly.

document_chunks

Used by ChunkRepository. Important fields:

- id: chunk id.
- document_id: foreign key to documents.id.
- content: chunk text.
- chunk_index: original chunk position.
- embedding: vector.
- source_type: gmail or calendar.
- occurred_at: used in search filters, but insertChunks currently does not set it.

Important index: create an ANN/vector index on embedding, for example IVFFlat or HNSW depending on pgvector version. Also index document_id and source_type.

conversations

Used by ConversationRepository and MemoryService. Important fields:

- conversation_id: UUID string shared by multiple turns.
- user_id: owner.
- user_message: raw user text.
- assistant_message: assistant text or JSON string for structured email-agent interrupts.
- metadata: JSON string/JSONB with mode, source count, durations, etc.
- is_deleted: soft delete flag.
- created_at.

Design note: each row stores one user+assistant pair. This is simple but less flexible than storing each message as a separate row with role and content.

sync_logs

Used by SyncLogRepository and sync controller. Important fields: id, source, status, user_id, sync_started_at, sync_completed_at, documents_fetched, documents_stored, last_sync_timestamp, error_message.

llm_usage_logs

Used by StatsRepository.insertLLMPrice. Important fields: conversation_id, provider, model, input_tokens, output_tokens, input_cost, output_cost, invocation_type, user_id, created_at.

embedding_costs

Used by EmbeddingRepository.logEmbeddingCost and stats. Important fields: batch_id, model, document_count, total_tokens, estimated_cost, status, processed_at. Current active EmbeddingPipeline does not call logEmbeddingCost, so embedding-cost stats may remain empty.

recipients

Used by RecipientRepository.getRelavantRecipient for email agent recipient resolution. Important fields: id, user_id, email, name, given_name, family_name, source, interaction_count, last_interaction_at, is_favorite. It uses PostgreSQL trigram functions similarity and %, so pg_trgm extension is required.

agent_checkpoints

A PostgresCheckpointer file contains SQL comments for agent_checkpoints, but active graphs use MemorySaver, not this table. If productionizing agents, use a persistent checkpointer.

Schema design decisions

- Unified documents let Gmail and Calendar share one ingestion and RAG pipeline.
- JSON metadata keeps source-specific fields without needing separate email/event tables.
- Separate chunks table supports multiple embeddings per document and better retrieval than one vector per whole document.
- conversations rows are simple and easy to retrieve, but message-per-row would be more scalable.

- sync_logs make async work observable.
- llm_usage_logs supports cost tracking and dashboards.

What can be improved

- Add migrations and seed/setup docs.
- Enforce foreign keys and unique indexes, especially (user_id, document_id) and (user_id, source).
- Use consistent credential field names: choose scopes or scope, not both.
- Populate document_chunks.occurred_at during insert so date filters work.
- Persist LangGraph checkpoints.
- Add row-level ownership checks to all personal-data endpoints.
- Create a separate messages table or JSON transcript for richer chat history.
- Track embedding usage from the active chunk embedding path.

E. RAG System Detailed Explanation

What RAG means in this project

RAG means Retrieval-Augmented Generation. In this project, the LLM does not answer personal questions from memory alone. The backend first retrieves relevant chunks from the user's synced Gmail and Calendar data, then gives those chunks to the LLM as context. The LLM answers based on that context.

Why RAG is needed

The user's personal emails and calendar events are private, recent, and not present in the base model's training data. RAG allows the assistant to answer questions about that private data without fine-tuning a model. It also helps reduce hallucination because the prompt tells the model to answer only from retrieved context.

Complete ingestion pipeline

Files involved:

- backend/src/api/controllers/syncController.js
- backend/src/RAG/ingestion/ingestionPipeline.js
- backend/src/service/sources/GmailDataSource.js
- backend/src/service/sources/GoogleCalendarDataSource.js
- backend/src/service/normalizers/GmailNormalizer.js
- backend/src/service/normalizers/GoogleCalendarNormalizer.js
- backend/src/database/documentRepository.js
- backend/src/RAG/ingestion/embeddingPipeline.js
- backend/src/RAG/ingestion/chunker.js
- backend/src/RAG/ingestion/embeddingsProvider.js
- backend/src/database/chunkRepository.js

Step-by-step:

1. User starts sync through /sync/gmail or /sync/calendar.
2. SyncController creates a sync log and returns immediately.
3. performDocumentsSync runs in the background.
4. IngestionPipeline.runIngestion selects source and normalizer from the SOURCES map.
5. For full sync, it fetches up to 500 records. For incremental sync, it uses last successful sync time, or last 7 days if none exists.
6. Gmail fetch uses Gmail query after:YYYY/MM/DD; Calendar fetch uses timeMin.
7. Normalizers create unified document objects.
8. Existing documents are skipped by findByDocumentId(doc.documentId, userId).
9. New documents are inserted into documents with needs_embedding expected to be true by DB default.
10. EmbeddingPipeline finds pending documents for the user.

11. Each document is chunked with RecursiveCharacterTextSplitter.
12. Each chunk is embedded using OpenAI embeddings.
13. Chunk rows are inserted into document_chunks.
14. The document is marked as not needing embedding.

Document upload flow

There is no user document upload feature yet. In this codebase, ingestion comes from external APIs: Gmail and Google Calendar. The frontend has an attach icon in the composer, but no upload route or file parser exists.

Text extraction flow

Gmail extraction:

- GmailNormalizer.extractHeaders extracts From, To, Subject, and Date.
- extractContent handles text/plain, text/html, and multipart payloads.
- HTML is converted to plain text by regex stripping scripts/styles/tags and decoding common HTML entities.
- cleanContent collapses excessive newlines, removes signature-like trailing content, and truncates above 32000 chars.

Calendar extraction:

- GoogleCalendarNormalizer skips cancelled events.
- It handles all-day events and dateTime events.
- It extracts summary, start, end, location, description, attendees, organizer, recurrence, html_link, and conference data.
- buildContent creates a plain-text block designed for embeddings.

Chunking strategy

backend/src/RAG/ingestion/chunker.js uses RecursiveCharacterTextSplitter with:

- chunkSize: 2700 characters.
- chunkOverlap: 400 characters.

This means each chunk is big enough to contain meaningful context, while overlap reduces the chance that important meaning is split across chunk boundaries. For emails and calendar events, this is reasonable because many items are short but long email threads can still be split.

Embedding generation

Embedding in embeddingsProvider.js uses OpenAIEmbeddings with:

- model from OPENAI_EMBEDDING_MODEL or default text-embedding-3-small.
- API key from OPENAI_API_KEY.
- dimensions from EMBEDDING_DIMENSIONS or default 1536.

embedChunks sends chunk contents to embedDocuments and attaches vectors back to chunk objects. embedQuery embeds one user query for retrieval.

Vector storage

Chunk vectors are inserted into document_chunks.embedding as pgvector-compatible strings like [0.1,0.2,...]::vector. Retrieval uses PostgreSQL vector distance:

```
ORDER BY c.embedding <=> $1::vector
```

ChunkRepository.searchByEmbedding joins chunks to documents and scopes search by d.user_id = \$2, which is important for privacy.

Metadata stored with chunks

The chunk row stores content, chunk_index, embedding, source_type, and maybe occurred_at. Source metadata is read through the joined document row: document id, external source id, author, and metadata JSON. The metadata contains Gmail message id/thread id/subject/from/to/date, or Calendar event id/start/end/attendees/organizer/location/etc.

Retrieval flow

Retriever.retrieve does:

1. Validate query and userId.
2. Embed query with OpenAI embeddings.
3. Search chunks for that user by vector distance.
4. Return empty list if no chunks are found.
5. Filter chunks with distance ≤ 0.6 .
6. Return filtered chunks.

The hard threshold is a simple relevance control. It should be tuned with real evaluation data.

Context creation

buildContext in contextBuilder.js formats each chunk as a numbered source:

```
[Source 1] gmail 2026-05-01 from sender@example.com - Subject  
<chunk content>
```

It approximates token budget as 4 characters per token and uses a default max of 4000 tokens. It always includes at least one block if chunks exist.

Prompt creation

prompts.js has a system prompt that says the assistant has access to the user's personal data, should answer using context, cite sources, ask clarifying questions when ambiguous, and avoid hallucination when no context exists.

buildPrompt creates messages:

- System prompt.
- Prior conversation history.
- User message containing retrieved context plus the question.

LLM call

LLMService.generateResponse supports two providers: OpenAI and Anthropic. It creates ChatOpenAI and ChatAnthropic clients from env vars. It invokes the selected model, reads llmResponse.content, stores the model name, duration, token counts if usage_metadata exists, and logs estimated costs into llm_usage_logs.

Final answer generation

QueryPipeline.run returns { answer, sources, conversationId, model }. RagChain.chat wraps this into an API-friendly response with sourcedDocuments containing chunk content, document id, source type, and metadata.

Error cases

- Missing userId throws in RagChain.
- Missing conversationId throws in QueryPipeline.
- Invalid llmProvider throws in LLMService.
- No chunks returns no-context prompt rather than a crash.
- Embedding provider failure fails retrieval.
- Database vector extension missing will fail search.
- Embedding dimension mismatch between stored vectors and query vectors will fail.
- Stats logging failure is caught and logged without failing the answer.

How hallucination is reduced

- The system prompt tells the model to answer from context and say when context is missing.
- Retrieved context is source-numbered.
- The answer should cite sources.
- Vector retrieval is user-scoped.
- Distance filtering removes weak matches.

This reduces hallucination but does not eliminate it. Stronger improvements would include answer-grounding checks, source citation validation, hybrid search, reranking, and tests with expected answers.

Why RAG instead of fine-tuning

Fine-tuning is not a good fit for constantly changing private Gmail and Calendar data. It would be expensive, slow to update, and risky for privacy. RAG keeps data in the database, updates quickly after sync, and can cite retrieved evidence.

Alternatives

- Simple keyword search: easier, but misses semantic matches.
- Full-text search: good for exact words and ranking, but weaker for paraphrases.
- Hybrid search: combines full-text and vector retrieval; likely a strong next improvement.
- Reranking: retrieves many chunks then reorders them with a cross-encoder or LLM; improves precision.
- Fine-tuning: better style or task behavior, not good for constantly changing private facts.
- Managed vector DB: better scale/ops for vectors, but more infrastructure.

How to explain the RAG pipeline in an interview

Say: I built RAG in two stages. First, sync and indexing: I pull Gmail and Calendar data, normalize it into documents, split content into overlapping chunks, create embeddings, and store chunk vectors in Postgres. Second, query-time retrieval: I embed the user question, run vector search scoped to the user's chunks, build a source-cited context block, combine it with conversation history, call the LLM, save the conversation, and return the answer with sources.

Likely RAG interview questions and answers

- Question: Why chunk documents?
Answer: Because embeddings work better on focused pieces of text. Whole emails or long threads may contain multiple topics, so chunking improves retrieval precision.
- Question: Why overlap chunks?
Answer: Overlap prevents important context from being lost at chunk boundaries.
- Question: Why store metadata?
Answer: Metadata lets the answer cite where information came from and lets the UI show source type, author, subject, or event details.
- Question: How do you prevent one user from seeing another user's data?
Answer: The retrieval query joins chunks to documents and filters by documents.user_id. Production should also require auth and derive userId from JWT.
- Question: What is the retrieval threshold?
Answer: The current code filters chunks with distance ≤ 0.6 . That is a heuristic and should be tuned using evaluation data.
- Question: What happens when no relevant context is found?
Answer: The context builder says no relevant context was found, and the prompt instructs the model to say it cannot find enough information instead of hallucinating.
- Question: Why not just use the LLM without RAG?
Answer: The LLM does not know the user's private emails or calendar events, and it should not guess. RAG gives it the needed private context at query time.

F. Agentic AI Flow Detailed Explanation

What agentic AI means here

In this project, agentic AI means the backend does not only generate text. It runs a stateful workflow that can collect missing information, call tools/APIs, pause for user decisions, and perform an external action only after safety checks.

Intent router

File: `backend/src/agent/intentRouter.js`.

The intent router calls `LLMService.generateResponse` with a classifier prompt. Valid active intents are `calendar_agent`, `email_draft`, `email_reply`, `email_read`, and `rag`. If output is not valid or classification fails, it returns `general`. The prompt also mentions `calendar_rag` and `general`, but `calendar_rag` is not in `VALID_INTENTS`. This means calendar read questions likely return `rag` or `general`, not `calendar_rag`.

Calendar agent

Files:

- `backend/src/agent/calenderAgent/state.js`
- `backend/src/agent/calenderAgent/nodes.js`
- `backend/src/agent/calenderAgent/graph.js`
- `backend/src/service/sources/GoogleCalendarDataSource.js`

What it does: creates Google Calendar events after collecting details, checking conflicts, and getting confirmation.

Why an agent is needed: Calendar creation is multi-step. A normal API call expects complete structured input, but users say things like schedule a call tomorrow afternoon. The agent can extract fields, ask missing questions, check conflicts, suggest slots, and wait for confirmation.

State fields:

- `userId`
- `userMessage`
- `eventDetails`: title, date, startTime, endTime, description, attendees, location
- `missingFields`
- `conflicts`
- `suggestedSlots`
- `confirmationStatus`
- `responseToUser`
- `messages`

Graph nodes:

- `parse_intent`: calls Anthropic Claude Haiku to extract event details as JSON.

- `ask_for_missing_info`: asks one required missing field at a time.
- `check_conflicts`: lists primary calendar events in the requested time window.
- `suggest_slots`: shows up to three free slots using Calendar freebusy.
- `await_confirmation`: returns a preview and asks yes/no.
- `create_event`: inserts the event into the user's primary Google Calendar.

Tools/APIs available to this agent:

- `getGoogleCalendarClient(userId)`: gets an authenticated Google Calendar API client.
- `calendar.events.list`: checks conflicts.
- `calendar.freebusy.query`: finds free slots between 8 AM and 8 PM.
- `calendar.events.insert`: creates the final event.

How confirmation is handled: ChatController passes `confirmationStatus` from the frontend. If status is confirmed, the graph routes directly to `create_event`. If rejected, the graph ends.

Memory/state: The graph uses `MemorySaver`, so state persists only inside the running backend process. `PostgresCheckpoint` exists but is not used.

Safety checks:

- Required fields are collected before event creation.
- Calendar conflicts are checked before confirmation.
- User confirmation is required before insertion.

Limitations:

- Only creation is implemented even though the intent prompt mentions update/delete/cancel.
- LLM JSON parsing can fail and silently become empty extraction.
- Date/time parsing depends on the LLM and current server date.
- Timezone is hardcoded to Asia/Kolkata.
- State is not persisted across restarts.
- Standalone `/agent/calender` route is not mounted.

Interview explanation: I used a LangGraph state machine because calendar creation is a controlled workflow. The graph lets me model each step, return to the user when information is missing, call Calendar APIs only at the right point, and require confirmation before side effects.

Email agent

Files:

- `backend/src/agent/emailAgent/graph.js`
- `backend/src/agent/emailAgent/state.js`
- `backend/src/agent/emailAgent/index.js`
- `backend/src/agent/emailAgent/tools.js`
- `backend/src/agent/emailAgent/nodes/parseRequest.js`

- backend/src/agent/emailAgent/nodes/resolveRecipient.js
- backend/src/agent/emailAgent/nodes/presentRecipientChoice.js
- backend/src/agent/emailAgent/nodes/draftEmail.js
- backend/src/agent/emailAgent/nodes/reviewDraft.js
- backend/src/agent/emailAgent/nodes/prepareSend.js
- backend/src/agent/emailAgent/nodes/revokeWindow.js
- backend/src/agent/emailAgent/nodes/sendEmail.js
- backend/src/service/email/gmailSendService.js

What it does: creates a safe email workflow from natural language. It extracts intent, resolves recipient, drafts the email, lets the user review/edit/cancel, waits through a revoke window, and then sends through Gmail.

Why an agent is needed: Email sending is a high-impact side effect. A one-shot API could accidentally send the wrong content to the wrong person. This graph forces human-in-the-loop confirmation and gives a short revoke period.

State fields:

- user_prompt and original_user_request
- purpose and tone
- recipient_name and recipient_email_from_request
- recipient_candidates and chosen_recipient
- current_draft, previous_draft, draft_history, edit_instructions
- approval_status
- send_status
- approval_timestamp, revoke_deadline, pending_send_token
- message_id, thread_id
- cancelled, last_error, final_response

Graph nodes:

- parse_request: OpenAI structured output extracts recipient name/email, tone, and purpose.
- resolve_recipient: uses explicit email or searches recipients by name.
- present_recipient_choice: LangGraph interrupt asks the user to choose or enter a recipient.
- draft_email: OpenAI structured output creates subject and body.
- review_draft: interrupt asks the user to approve, edit, regenerate, or cancel.
- prepare_send: creates a pending send token and 6-second revoke deadline.
- revoke_window: interrupt lets the user revoke before timeout.
- send_email: calls Gmail send API after safety checks.

Tools/APIs available:

- OpenAI structured output for parsing and drafting.
- RecipientRepository.getRelavantRecipient for recipient search.
- Gmail API through sendEmail in gmailSendService.js.

- LangGraph interrupts and Command resume.

How tool calling works: This graph does not use LLM tool calling in the OpenAI function-call sense. Instead, the workflow itself calls deterministic tools/services at specific nodes. This is safer because the graph controls when recipient search and Gmail send can happen.

How the agent decides what to do: The graph edges route based on state. After recipient choice, cancelled ends or draft_email runs. After review, approval goes to prepare_send, edit goes back to draft_email, and cancel ends. After revoke window, sending goes to send_email; otherwise it ends.

How user confirmation is handled: interrupt() pauses the graph and returns structured payloads to the backend. The frontend renders cards for recipient choice, draft approval, and pending send. User actions are sent back as normal chat messages, and invokeEmailAgent resumes the graph with Command({ resume }).

Revoke window: prepareSend.js sets REVOKE_WINDOW_MS = 6000. index.js schedules a timer that resumes the graph with { action: 'timeout', token }. If the user sends revoke/cancel/undo/stop before timeout, the pending timer is cleared.

Safety checks:

- Recipient must be chosen or entered as a valid email.
- Draft must be approved before sending.
- Email cannot be sent early during the revoke window.
- Pending send token is checked.
- sendEmailNode requires approval_status approved, send_status sending, and expired revoke deadline.
- Errors from Gmail send return failed status instead of throwing to the user.

Limitations:

- Graph state is in process memory. A server restart loses active email sessions.
- The recipient repository method name is misspelled getRelavantRecipient.
- Recipient search needs pg_trgm extension.
- Reply-to-existing-email is disabled in ChatController because original thread verification is not safe yet.
- The 6-second revoke window is useful for a demo but should be configurable.
- The pending timers map is in memory and not durable across restarts or multiple server instances.

Interview explanation: I designed the email agent as a human-in-the-loop LangGraph workflow. The LLM helps parse and draft, but the graph owns safety. It cannot send until the user selects a recipient, approves the draft, and passes a revoke window.

G. LangChain, LangGraph, and AI Libraries

AI-related backend packages

- @langchain/openai: Used for ChatOpenAI in LLMService, intent routing, email agent structured output, and OpenAIEmbeddings.
- @langchain/anthropic: Used for ChatAnthropic in LLMService and directly in calendar agent nodes.
- @langchain/langgraph: Used for StateGraph, MemorySaver, interrupts, Command, and StateSchema in agents.
- @langchain/textsplitters: Used for RecursiveCharacterTextSplitter in the RAG chunker.
- @langchain/core: Used for prompts, messages, chat history, and LangGraph-related types.
- langchain: General LangChain package dependency.
- openai: Installed as underlying OpenAI dependency.
- @google/generative-ai: Installed but not used in active inspected code.
- natural: Used in textProcessing utilities for tokenization.
- stopword: Used in textProcessing utilities to remove stop words.
- zod: Used by email agent state schemas and structured output schemas.

How model calls are made

LLMService creates two chat model clients in its constructor:

- ChatOpenAI with env model, temperature, maxTokens, retries, timeout, and streaming false.
- ChatAnthropic with env model, temperature, maxTokens, retries, timeout, and streaming false.

generateResponse chooses provider by string. It accepts only OpenAI or Anthropic. It calls llm.invoke(messages) and reads content plus usage_metadata.

Structured output

The email agent uses .withStructuredOutput(zodSchema) for two tasks:

- Parse email request into recipient_name, recipient_email, tone, purpose.
- Draft email into subject and body.

This is stronger than asking the model for plain JSON because LangChain validates the model output against Zod schemas.

Tools bound to model

The code does not bind tools directly to a model with function/tool calling. Instead, tools are ordinary functions called by graph nodes. This is simpler and more controlled.

Prompt management

Prompts are stored in code:

- RAG prompts in backend/src/RAG/query/prompts.js.
- Intent classifier prompt in backend/src/agent/intentRouter.js.
- Calendar extraction prompt inside backend/src/agent/calenderAgent/nodes.js.
- Email parse/draft prompts in email agent node/tool files.

For production, prompts could be versioned and tested separately.

Token and cost tracking

LLMService reads usage_metadata and logs input/output token counts. It estimates cost using hardcoded OpenAI-like prices and converts USD to INR by calling <https://api.frankfurter.app/latest?from=USD&to=INR>. This is useful, but it is not provider/model-specific enough yet. Anthropic pricing differs, and exchange-rate network calls add latency and failure risk.

Improvements

- Make pricing model-specific and provider-specific.
- Cache exchange rates.
- Use LangSmith or structured tracing for RAG and agent runs.
- Add prompt tests and golden-answer evaluation.
- Implement streaming support or remove the streaming route.
- Add reranker and query transformer, since files exist but are empty.

H. Frontend Explanation

Frontend framework

The frontend uses React 19.2 with Vite. The Vite package is rolldown-vite through an npm alias. State management uses Zustand. Markdown rendering uses react-markdown. Real-time sync progress uses socket.io-client.

Why this stack is acceptable

React and Vite are good for a fast SPA. Zustand is simpler than Redux and works well for this app because stores are small: auth, chat, and sync. Plain fetch clients are enough for the current API size.

Routing

App.jsx manually maps paths to pages:

- / -> home
- /chat -> chat
- /stats -> stats
- /settings -> settings
- /profile -> profile
- /login -> login
- /auth/callback -> auth-callback

It pushes browser history manually and listens to popstate. frontend/vercel.json rewrites all routes to index.html, which supports direct SPA page loads.

State management

- authStore.js: user, isAuthenticated, isLoading, error, setUser, logout.
- chatStore.js: messages, typing state, conversation id, agent active flags, pending confirmation, conversations list, sendMessage, load history, email status sync, reset/start chat.
- syncStore.js: Gmail and Calendar sync progress states and actions.

API clients

- auth.js: Google login, JWT get/set/remove, current user, logout. Default base URL 9000.
- chat.js: send message, get email status, create conversation, get history, list conversations. Default base URL 9000.
- sync.js: start Gmail/Calendar sync. Default base URL 9000.
- stats.js: stats dashboard. Default base URL 2020.
- home.js: daily summary and upcoming events with dummy fallback. Default base URL 2020.
- user.js: update display name. Default base URL 2020.

There is inconsistency in default API ports: some clients default to 9000, others to 2020, while backend index.js defaults to 2020. In real deployment, VITE_API_BASE_URL should be set consistently.

Major pages

- LoginPage: Google login, guest mode, animated design.
- AuthCallbackPage: reads token from URL, stores it, loads user, navigates to chat.
- HomePage: greeting, query composer, suggestions, dummy/fallback daily summary and upcoming events.
- ChatPage/ChatWindow: main assistant UI, message list, composer, source pills, calendar confirmation, email agent cards.
- ProfilePage: profile data, display-name edit, Gmail and Calendar sync buttons, Socket.IO progress panels, sync history, logout.
- StatsPage: activity dashboard with SVG charts.
- SettingsPage: mostly dummy local settings UI, not wired to backend except theme passed from App.

Chat UI flow

ChatWindow lets the user type a message. `sendMessage` in `chatStore` optimistically appends the user message, calls backend, then appends AI response. It handles three kinds of responses:

- Normal text response.
- Calendar agent response with `pendingConfirmation` and mode agent.
- Email agent structured response types: `recipient_choice`, `draft_approval`, `pending_send`.

Email agent frontend flow

- `recipient_choice` renders a card with candidates and email input.
- `draft_approval` renders a draft preview and Approve/Edit/Regenerate/Cancel buttons.
- `pending_send` renders countdown state and Revoke send button.
- `syncEmailStatus` polls `/chat/email-status/:conversationId` to update pending send status.

Sync frontend flow

ProfilePage connects to Socket.IO with `socketService.connect(userId)`, listens for `sync:gmail:*` and `sync:google_calendar:*`, and updates `syncStore`. It filters events by `syncId` refs. Sync buttons use `fetch` directly in ProfilePage rather than `syncApi`.

Loading/error states

- Chat has `isTyping` and error messages.
- Auth callback has `processing/success/error` states.
- Sync progress panels show `progress, complete, and error` states.
- Stats API returns empty data on failure so UI shows `no-data` states.
- Sidebar shows `conversation loading/error/empty` states.

Frontend gaps

- No React Router, so route handling is manual.
- Socket service is mainly used on ProfilePage, not globally.
- Chat streaming is not implemented.
- Attach and voice buttons are UI-only.
- Settings page is mostly dummy and not backed by APIs.
- Home page daily summary/upcoming events call endpoints that do not exist and fall back to dummy data.
- Sidebar groups conversations by updatedAt/createdAt, but backend returns startedAt/lastMessageAt, so grouping can be wrong.

How to explain frontend briefly

The frontend is a React/Vite SPA with Zustand stores. It has pages for login, home, chat, profile, stats, and settings. The chat UI calls the backend chat endpoint and can render normal RAG answers, calendar confirmations, and structured email-agent cards. The profile page starts Gmail/Calendar sync and listens to Socket.IO progress events.

I. Complete Feature-by-Feature Flow

Google login and account connection

- User clicks Continue with Google in frontend/src/pages/LoginPage.jsx.
- `authApi.loginWithGoogle` calls GET `/auth/google/login`.
- The backend creates a Google OAuth URL with Gmail and Calendar scopes.
- Google redirects to `/auth/google/callback` with a code.
- `AuthController` exchanges the code, reads Google profile data, upserts users, encrypts access and refresh tokens, stores both gmail and `google_calendar` credentials, signs a JWT, and redirects to the frontend callback.
- `AuthCallbackPage` stores the JWT as `myra_auth_token`, calls `/auth/me`, then puts the user in `authStore`.
- Edge cases: denied consent, missing code, expired/invalid JWT, missing `TOKEN_ENCRYPTION_KEY`.
- Interview explanation: this is OAuth-based authentication plus integration authorization. The app never gives raw Google tokens to the browser.

Gmail data sync

- User clicks Sync Gmail Now in `ProfilePage`.
- The frontend posts `userId` and `syncType` to `/sync/gmail`.
- `SyncController` creates a `sync_logs` row and immediately returns `syncId`.
- `performDocumentsSync` emits progress over `Socket.IO`.
- `IngestionPipeline` chooses `GmailDataSource` and `GmailNormalizer`.
- `GmailDataSource` gets a valid access token, refreshes if needed, lists Gmail messages, and fetches each message in batches.
- `GmailNormalizer` extracts headers, decodes text/plain or text/html, recursively handles multipart messages, cleans content, and creates a document object.
- `DocumentRepository` stores new documents in `documents` and skips already-seen `document_id` values for that user.
- `EmbeddingPipeline` splits each document, embeds chunks, stores them in `document_chunks`, and marks the document as embedded.
- Edge cases: empty emails skipped, duplicate documents skipped, token refresh failure, Google API rate limits, embedding failure, missing auth enforcement on sync route.

Calendar data sync

- User clicks Sync Calendar in `ProfilePage` or `Sidebar`.
- Frontend posts to `/sync/calendar`.
- `SyncController` uses `IngestionPipeline` with `sourceName` calendar.
- `GoogleCalendarDataSource` gets an authenticated calendar client and lists primary calendar events.
- `GoogleCalendarNormalizer` skips cancelled events, handles all-day and `dateTime` events, extracts attendees, organizer, location, description, recurrence, and builds embedding-friendly content.
- Documents are stored in `documents` and later chunked/embedded in `document_chunks`.
- Edge cases: cancelled events return null, missing start/end data may break normalization, and full sync uses a hardcoded 2026-01-01 `timeMin`.

RAG chat over personal data

- User types a personal-data question in ChatWindow.
- chatStore.sendMessage calls POST /chat/message.
- ChatController validates message and resolves userId from JWT or SYNC_USER_ID.
- routeIntent classifies the message using an OpenAI-backed LLMService call.
- For rag, email_read, or calendar_rag intents, RagChain.chat is used.
- QueryPipeline embeds the query with OpenAIEmbeddings, retrieves chunks with pgvector distance, filters by distance <= 0.6, builds a context block with source labels, loads conversation history, builds the final prompt, calls LLMService, saves the turn, and returns answer plus source documents.
- Frontend renders the response and source count in the assistant bubble.
- Edge cases: no relevant chunks results in a no-context block; no userId throws; invalid llmProvider throws; token/cost stats can fail without failing the chat.

General chat

- If routeIntent returns general, ChatController does not use RAG.
- It loads conversation history, creates a system message for a helpful assistant, and calls LLMService directly.
- The response is saved with metadata mode general_chat.
- This is useful because not every question should search private data.
- Edge case: intent classifier can misclassify; default provider must be exactly OpenAI or Anthropic.

Calendar event creation agent

- User asks to schedule, create, update, cancel, or delete a calendar event. routeIntent returns calendar_agent.
- ChatController invokes calendarAgentGraph with a stable thread_id equal to the conversationId.
- parse_intent uses Anthropic Claude Haiku to extract title, date, startTime, endTime, description, attendees, and location as JSON.
- If required fields title, date, or startTime are missing, ask_for_missing_info returns a targeted question.
- When enough fields are present, check_conflicts lists calendar events in the target time window.
- If conflicts exist, suggest_slots calls freebusy and shows free one-hour slots.
- If no conflict exists, await_confirmation returns a preview and asks yes/no.
- On confirmationStatus confirmed, the graph creates the Google Calendar event with sendUpdates all when attendees exist.
- Edge cases: date parsing depends on the LLM, timezone conversion uses Asia/Kolkata, graph checkpoints are in memory only, and rejection ends without a custom rejection message.

Secure email drafting and sending agent

- User asks to compose/write/send a new email. routeIntent returns email_draft.
- ChatController starts the emailAgent graph with thread_id secure_email_<userId>_<conversationId>.
- parse_request uses OpenAI structured output to extract recipient name/email, tone, and purpose.

- `resolve_recipient` either trusts an explicit valid email or searches recipients using trigram similarity in `RecipientRepository`.
- `present_recipient_choice` interrupts and asks the user to choose a candidate or enter an email.
- `draft_email` uses OpenAI structured output to produce subject and body.
- `review_draft` interrupts with a draft card. The user can approve, edit, regenerate, or cancel.
- `prepare_send` creates a 6-second revoke window with a random `pending_send_token`.
- `revoke_window` interrupts and the frontend can send revoke. If timeout occurs, the graph moves to `send_email`.
- `send_email` uses Gmail API to send a plain-text RFC 2822 message.
- Edge cases: `MemorySaver` means email sessions vanish on server restart, recipient search needs `pg_trgm` similarity support, reply drafting is intentionally disabled until original thread verification is implemented.

Stats dashboard

- `StatsPage` calls `statsApi.getAll(range)`.
- `StatsController` fetches LLM usage, embedding usage, chat sessions, Gmail document counts, and Calendar document counts.
- Frontend draws KPI tiles and pure SVG charts from returned arrays.
- If any fetch fails, `statsApi` returns empty arrays so the page remains usable.
- Edge cases: stats API default base URL is 2020 while other APIs default to 9000; chart labels are generated from array length and not actual row dates.

J. Tech Stack and Tools

Backend technologies

- Node.js with ES modules.
- Express 5.2.1 for HTTP API.
- PostgreSQL via pg.
- pgvector-style vector search with `<=>` operator.
- Socket.IO for progress events.
- Google APIs for OAuth, Gmail, and Calendar.
- JWT for app auth tokens.
- AES-256-CBC through Node crypto for OAuth token encryption.
- LangChain and LangGraph for LLM workflows.
- Zod for email-agent state and structured output validation.
- Nodemailer for alert emails.
- node-cron for cost alert scheduling.
- TypeScript compiler with `allowJs` for mixed JS/TS source.

Frontend technologies

- React 19.2.
- Vite/Rolldown Vite.
- Zustand for state.
- socket.io-client for sync progress.
- react-markdown for assistant response rendering.
- Custom CSS in `index.css/App.css` and style JS files.
- Vercel SPA rewrite config.

AI models

Model names are env-driven in most places. The stats controller includes colors for `claude-haiku-4-5-20251001`, `claude-3-5-sonnet-20241022`, and `gemini-embedding-001`. The active embedding provider defaults to `text-embedding-3-small`, while the stats controller still references Gemini embedding cost labels. Calendar agent nodes directly use Claude Haiku 4.5 as a hardcoded model.

Testing tools

The backend has ad-hoc node scripts in `backend/test`. There is no Jest/Vitest/Mocha setup. The frontend has ESLint but no test runner. `backend/package.json` maps `npm test` to `node test/test-setup.js`, which only tests `logger/config/schema` validation basics.

Deployment tools

Frontend has vercel.json for SPA rewrites. Backend has build/start scripts but no deployment manifest such as Dockerfile, Render/Railway config, or CI file in the inspected repo.

K. Design Decisions

Why this backend structure

The backend structure separates HTTP concerns from business logic. Controllers handle request validation and responses, services handle integrations and workflows, and repositories handle SQL. This makes the backend easier to explain and debug.

Why PostgreSQL

PostgreSQL handles relational application data and vector search in one database. For a personal assistant MVP, this reduces operational complexity.

Why the RAG pipeline

RAG is better than fine-tuning for personal data because user data changes daily and must remain private. The pipeline is also explainable: synced data -> chunks -> embeddings -> vector retrieval -> LLM answer.

Why this agent design

Calendar and email actions require multiple steps and user confirmation. LangGraph gives explicit state, transitions, and interrupts. This is safer than allowing an LLM to directly call send/create tools whenever it wants.

Why current API design

The API is simple REST. /chat/message acts as an orchestrator endpoint. /sync/* endpoints start background jobs. /stats/all aggregates dashboard data. This is practical for a small product, though production should split some concerns and harden auth.

Tradeoffs

- Optional auth makes local development easier but is unsafe for personal data.
- Fire-and-forget sync is simple but not durable like a queue.
- MemorySaver is simple but loses agent state on restart.
- Raw SQL is transparent but missing migrations make setup fragile.
- Hardcoded retrieval threshold is easy but needs evaluation.
- One chat endpoint is convenient but can become large as modes grow.

L. Bugs, Gaps, and Production Improvements

Current bugs or incomplete parts

- Route `/chat/message/stream` calls `chatController.sendMessageStream`, but that method is commented out. This endpoint will fail.
- `backend/src/api/routes/agent.js` is not mounted in `app.js`, so `/agent/calender` is unreachable.
- `CronManager` references undefined `jobs` embedding, `gmailSync`, and `calendarSync`.
- `CronManager` is not started in `index.js`.
- `GoogleAuthService.revokeAccess` calls `credentialsRepo.delete(credential.id)`, but `delete` expects a source, not an id.
- `Credential repository` mixes scope and scopes column names and has old upsert logic on (source) alongside newer upsert on (user_id, source).
- Some frontend API clients default to port 9000 and others to 2020.
- Sidebar conversation grouping reads `updatedAt` or `createdAt`, but backend returns `startedAt` and `lastMessageAt`.
- Home page calls `/stats/daily-summary` and `/calendar/upcoming`, but those backend endpoints do not exist.
- Settings page has many dummy values and TODOs.
- `RAG queryTransformer.js` and `reranker.js` are empty.
- Embedding cost logging exists but active chunk embedding pipeline does not call it.
- `document_chunks` insert does not set `occurred_at`, but retrieval filters support `occurredAfter/occurredBefore`.
- Sync routes accept `userId` from request body and do not enforce JWT auth.
- Chat/stat routes use optional auth and fallback `SYNC_USER_ID`.
- `backend/test/test-agent.js` imports `../src/service/router/intentRouter.js`, which does not exist. The actual file is `src/agent/intentRouter.js`.
- Google Calendar full sync starts at hardcoded `2026-01-01T00:00:00Z`.
- Gmail full sync default query is hardcoded `after:2026/01/01`.
- Calendar and email agent checkpoints are in memory only.
- Email pending-send timers are in memory only and will not work reliably across restarts or multiple instances.

Security improvements

- Require JWT auth for all chat, sync, stats, profile, and personal-data endpoints.
- Derive `userId` from JWT instead of request body.
- Persist and verify OAuth state to prevent CSRF in the OAuth flow.
- Use secure `httpOnly` cookies or short-lived access tokens plus refresh strategy instead of long-lived JWT in `localStorage`.
- Add rate limiting and request size validation.
- Add audit logs for external side effects like email sending and event creation.
- Consider stronger token encryption with AES-GCM to provide authentication, not only confidentiality.

Reliability improvements

- Use a real job queue for sync and embedding, such as BullMQ, pg-boss, or cloud queues.

- Make sync idempotent with unique (user_id, document_id) constraints.
- Persist LangGraph checkpoints and pending email timers.
- Add retries/backoff for Google API and embedding calls.
- Add health checks for DB, Google credentials, and model providers.
- Add structured logs with request ids and trace ids.

RAG quality improvements

- Add hybrid retrieval with PostgreSQL full-text search plus vector search.
- Implement reranker.js.
- Implement queryTransformer.js for follow-up question rewriting or multi-query retrieval.
- Tune chunk size and distance threshold with evaluation data.
- Add citation validation to ensure answers cite only retrieved sources.
- Add source filters from frontend, such as Gmail only or Calendar only.
- Store and use occurred_at on chunks.

Developer-experience improvements

- Add migrations and schema docs.
- Add .env.example.
- Add API tests with supertest.
- Add unit tests for normalizers, chunking, retrieval filtering, and graph routing.
- Add CI for typecheck, lint, tests, and build.
- Remove dead/old files or clearly mark them as planned.

M. Interview Question Bank

What is the main backend responsibility?

The backend authenticates users, syncs data from Google services, stores documents and embeddings, runs RAG queries, orchestrates agents, and exposes REST APIs to the frontend.

How does OAuth work in your project?

The frontend asks the backend for a Google consent URL. Google returns a code to the backend callback. The backend exchanges the code for tokens, fetches the Google profile, stores encrypted tokens, creates a user, signs a JWT, and redirects the frontend with that JWT.

How do you store private Google tokens?

Access and refresh tokens are encrypted in GoogleAuthService with AES-256-CBC using `TOKEN_ENCRYPTION_KEY` before being stored in `api_credentials`.

How do you refresh expired Google tokens?

`GoogleAuthService.isValidAccessToken` decrypts tokens, checks expiry with a 5-minute buffer, uses the refresh token if needed, updates the stored encrypted access token, and returns a valid access token.

What is your unified document model?

Emails and calendar events are normalized into a common shape: document id, source, type, content, title, timestamp, author, and metadata. This lets one RAG pipeline work across sources.

How does Gmail normalization work?

It extracts headers, decodes `base64url` body content, prefers plain text, strips HTML when needed, handles multipart recursively, cleans the text, truncates very long messages, and returns a gmail document.

How does Calendar normalization work?

It skips cancelled events, handles all-day and timed events, extracts attendees and organizer, builds a readable text representation, and stores source-specific metadata.

Why did you use chunk embeddings instead of document embeddings?

Chunks improve precision because long documents can contain multiple topics. Querying chunks returns the most relevant part instead of an entire document.

How is vector search done?

The query is embedded with `OpenAIEmbeddings`. PostgreSQL searches `document_chunks` by `pgvector` distance, joins to documents, filters by user id, optionally filters source/date, orders by vector distance, and limits topK.

How do you reduce hallucination?

I retrieve context from user data, format it as numbered sources, instruct the model to answer only from context, and tell it to say when context is missing.

Why use LangGraph?

LangGraph is useful for stateful workflows with branches and interruptions. Calendar and email actions are multi-step and require human confirmation, so an explicit graph is safer than one-shot generation.

How does the calendar agent check conflicts?

It creates start/end timestamps, calls Google Calendar events.list for that window, and if conflicts exist it calls freebusy to find available one-hour slots.

How does the email agent prevent accidental sends?

It resolves the recipient, drafts the message, interrupts for review, requires approval, creates a pending-send token, waits through a revoke window, and only then calls Gmail send after safety checks.

What are the main production gaps?

Auth is optional on several personal-data endpoints, migrations are missing, background jobs and graph checkpoints are in memory, streaming route is broken, and sync should use a durable queue.

How does the frontend connect to the backend?

API clients in frontend/src/api use fetch. chatStore calls /chat/message, authApi calls /auth/*, sync UI calls /sync/*, statsApi calls /stats/all, and ProfilePage uses Socket.IO for sync progress.

What would you improve first?

I would require auth everywhere and derive user id from JWT, add migrations and unique indexes, fix broken/unmounted routes, persist graph checkpoints, and add tests for RAG and agents.

N. File-by-File Revision Map

Backend files

- backend/index.js: Entrypoint: loads env, connects DB, starts server, attaches WebSocket, handles shutdown.
- backend/src/app.js: Express app config and route mounting.
- backend/src/config/environment.js: Config object and required DB env validation.
- backend/src/config/dbConfig.ts: Lazy PostgreSQL pool creation and connectToDB.
- backend/src/api/routes/authRoutes.js: Auth route paths.
- backend/src/api/controllers/authController.js: Google OAuth, user upsert, token storage, JWT, current user, display-name update.
- backend/src/api/routes/chat.js: Chat routes and optional auth.
- backend/src/api/controllers/chatController.js: Main chat orchestration across email agent, calendar agent, RAG, and general LLM.
- backend/src/api/routes/syncRoutes.js: Gmail/Calendar sync route paths.
- backend/src/api/controllers/syncController.js: Async sync orchestration, sync logs, ingestion, embedding, WebSocket events.
- backend/src/api/routes/stats.js: Stats route with optional auth.
- backend/src/api/controllers/statsController.js: Dashboard aggregation response shaping.
- backend/src/api/routes/agent.js: Standalone unmounted calendar-agent route.
- backend/src/database/userRepository.js: SQL for users table.
- backend/src/database/credentialRepository.js: SQL for API credentials and OAuth token storage.
- backend/src/database/documentRepository.js: SQL for documents and older document-level vector search.
- backend/src/database/chunkRepository.js: SQL for inserting/searching chunk embeddings.
- backend/src/database/conversationsRepo.js: Conversation persistence and sidebar conversation list queries.
- backend/src/database/syncLogsRepository.js: Sync log CRUD and stats.
- backend/src/database/statsRepository.js: LLM/embedding usage and dashboard aggregation queries.
- backend/src/database/recipientRepository.js: Recipient fuzzy search for email agent.
- backend/src/RAG/ragService.js: Top-level RAG chat service.
- backend/src/RAG/ingestion/ingestionPipeline.js: Selects source/normalizer and stores documents.
- backend/src/RAG/ingestion/embeddingPipeline.js: Finds pending docs, chunks, embeds, inserts chunks.
- backend/src/RAG/ingestion/chunker.js: RecursiveCharacterTextSplitter settings.
- backend/src/RAG/ingestion/embeddingsProvider.js: OpenAI embeddings wrapper.
- backend/src/RAG/retrieval/retriever.js: Query embedding plus vector retrieval and threshold filtering.
- backend/src/RAG/retrieval/contextBuilder.js: Formats retrieved chunks into source-numbered context.
- backend/src/RAG/query/queryPipeline.js: Full query-time RAG orchestration.
- backend/src/RAG/query/llmService.js: OpenAI/Anthropic model invocation and usage logging.
- backend/src/RAG/query/memoryService.js: Conversation history loading/saving.
- backend/src/RAG/query/prompts.js: RAG prompts and prompt builder.
- backend/src/RAG/retrieval/queryTransformer.js: Empty planned file.

- backend/src/RAG/retrieval/reranker.js: Empty planned file.
- backend/src/agent/intentRouter.js: LLM-based intent classifier.
- backend/src/agent/calenderAgent/state.js: Calendar agent state reducers.
- backend/src/agent/calenderAgent/nodes.js: Calendar agent node implementations.
- backend/src/agent/calenderAgent/graph.js: Calendar StateGraph wiring.
- backend/src/agent/calenderAgent/checkPointer.js: Unused Postgres checkpointer draft.
- backend/src/agent/emailAgent/graph.js: Email StateGraph wiring.
- backend/src/agent/emailAgent/state.js: Email state schema.
- backend/src/agent/emailAgent/index.js: Email graph invocation, serialization, interrupts, timers, status helpers.
- backend/src/agent/emailAgent/tools.js: Recipient search, email draft generation, Gmail send wrapper.
- backend/src/agent/emailAgent/nodes/*.js: Individual email workflow nodes.
- backend/src/service/sources/GmailDataSource.js: Gmail API fetcher.
- backend/src/service/sources/GoogleCalendarDataSource.js: Calendar API fetcher plus free-slot helpers.
- backend/src/service/normalizers/GmailNormalizer.js: Gmail raw message to document.
- backend/src/service/normalizers/GoogleCalendarNormalizer.js: Calendar raw event to document.
- backend/src/service/oauth/googleOAuthService.js: Token encryption, refresh, and OAuth helper methods.
- backend/src/service/email/gmailSendService.js: Raw RFC 2822 email building, Gmail send, Gmail draft save.
- backend/src/service/email/replyContextService.js: Planned Gmail thread lookup for replies using RAG.
- backend/src/service/websocket/sockeService.js: Socket.IO server singleton.
- backend/src/service/alertServices/CredAlertService.js: Monthly budget alert logic.
- backend/src/service/cron/credsAlertCron.js: node-cron job for cost alerts.
- backend/src/service/cron/cronManager.js: Incomplete cron manager.
- backend/src/utls/validation.js: Unified document and metadata validation helpers.
- backend/src/utls/logger.ts: Console logger.
- backend/src/utls/tokenCounter.js: Approximate token counting helpers.
- backend/src/utls/textProcessing.js: Keyword/entity utility functions.
- backend/src/utls/mailSender.js: Nodemailer wrapper.
- backend/src/utls/emailTemplates.ts: HTML cost-alert email builder.
- backend/src/utls/constants.ts: LLM invocation and sync constants.
- backend/src/utls/exchanceRates.ts: USD to INR API helper.

Frontend files

- frontend/src/App.jsx: Manual routing, theme, auth check, layouts.
- frontend/src/main.jsx: React root render.
- frontend/src/api/auth.js: Google login and JWT user session client.
- frontend/src/api/chat.js: Chat API client.
- frontend/src/api/sync.js: Sync API client.
- frontend/src/api/stats.js: Stats API client.
- frontend/src/api/home.js: Home dummy/fallback API client.

- frontend/src/api/user.js: Display name update client.
- frontend/src/store/authStore.js: Auth Zustand store.
- frontend/src/store/chatStore.js: Chat, agent, conversation Zustand store.
- frontend/src/store/syncStore.js: Gmail/Calendar sync progress store.
- frontend/src/service/socketService.js: Socket.IO client singleton.
- frontend/src/pages/LoginPage.jsx: Login screen and Google OAuth entry.
- frontend/src/pages/AuthCallbackPage.jsx: OAuth callback token handling.
- frontend/src/pages/HomePage.jsx: Home dashboard and prompt suggestions.
- frontend/src/pages/ChatPage.jsx: Chat page wrapper.
- frontend/src/components/chat/ChatWindow.jsx: Main chat UI and email/calendar cards.
- frontend/src/components/chat/Message.jsx: Older/general message component.
- frontend/src/components/chat/TypingIndicator.jsx: Typing indicator.
- frontend/src/components/chat/ChatInput.jsx: Placeholder, now embedded in ChatWindow.
- frontend/src/components/layout/Sidebar.jsx: Navigation, chat history, profile menu, calendar sync.
- frontend/src/pages/ProfilePage.jsx: Profile, data connections, sync progress, sync history.
- frontend/src/pages/StatsPage.jsx: Stats dashboard and SVG chart components.
- frontend/src/pages/SettingsPage.jsx: Mostly local/dummy settings UI.
- frontend/vite.config.js: Vite config with React plugin.
- frontend/vercel.json: SPA rewrite.
- frontend/eslint.config.js: ESLint config.
- frontend/tailwind.config.js: Tailwind config, though custom CSS dominates.

Final revision checklist

- Practice the one-minute and three-minute answers.
- Be ready to draw the RAG pipeline from sync to answer.
- Be ready to explain why RAG beats fine-tuning for private changing data.
- Be honest about gaps: auth hardening, migrations, queues, persistent checkpoints, and broken streaming route.
- Emphasize safety in the agents: confirmation before Calendar changes and approval/revoke before email sending.
- Use exact file paths from this document when asked where logic lives.